

연결 리스트를 이용한 다항식 표현 및 연산 정리

1. 다항식 표현 방식 비교

방식	특징
배열 기반	- 연속적인 메모리 사용 - 고정 크기 제한 있음 - 항 추가/삭제가 어려움
연결 리스트 기반	- 동적 메모리 사용 - 항 추가/삭제 용이 - 각 노드에 계수, 지수, 링크 필드 포함

중요 차이점: 배열은 고정된 크기의 메모리 공간을 사용하지만, 연결 리스트는 링크 필드로 노드를 연결하여 동적 생성이 가능하며, 마지막 노드의 링크는 **NULL**을 가짐.

2. 연결 리스트 기반 다항식 노드 구조

```
typedef struct Node {  
    int coef; // 계수  
    int exp;  // 지수  
    struct Node* link; // 다음 노드 포인터  
} Node;
```

3. 다항식 예시 표현

$a(x) = 3x^{12} + 2x^8 + 1 \rightarrow$ 연결 리스트 형태:

[3, 12] -> [2, 8] -> [1, 0] -> NULL

헤더 노드는 첫 노드를 가리키며, 각 노드는 계수와 지수 정보를 가짐.

4. 다항식 덧셈 알고리즘

기본 연산 흐름:

- 두 리스트 $a(x)$ 와 $b(x)$ 를 순회
- 지수 기준으로 비교
- 지수가 같으면 계수 더해 결과 리스트 $c(x)$ 에 추가
- 지수가 다르면 큰 지수 항을 결과 리스트에 복사 후 이동

연산 알고리즘 요약:

```

while (p != NULL && q != NULL) {
    if (p->exp == q->exp) {
        int sum = p->coef + q->coef;
        if (sum != 0) insertLast(&c, sum, p->exp);
        p = p->link;
        q = q->link;
    } else if (p->exp > q->exp) {
        insertLast(&c, p->coef, p->exp);
        p = p->link;
    } else {
        insertLast(&c, q->coef, q->exp);
        q = q->link;
    }
}
// 남은 항 처리
while (p != NULL) { insertLast(&c, p->coef, p->exp); p = p->link; }
while (q != NULL) { insertLast(&c, q->coef, q->exp); q = q->link; }

```

5. Header 구조체와 다항식 리스트 관리

```

typedef struct ListHeader {
    int length;        // 전체 노드 수
    Node* head;        // 첫 노드 포인터
    Node* tail;        // 마지막 노드 포인터
} ListHeader;

```

초기 생성 시:

```

ListHeader* createList() {
    ListHeader* plist = (ListHeader*)malloc(sizeof(ListHeader));
    plist->length = 0;
    plist->head = NULL;
    plist->tail = NULL;
    return plist;
}

```

노드 삽입 함수 예시:

```

void insertLast(ListHeader* plist, int coef, int exp) {
    Node* newNode = (Node*)malloc(sizeof(Node));
    newNode->coef = coef;
    newNode->exp = exp;
    newNode->link = NULL;

    if (plist->head == NULL) {

```

```
        plist->head = newNode;
    } else {
        plist->tail->link = newNode;
    }
    plist->tail = newNode;
    plist->length++;
}
```

6. 원형 연결 리스트 (Circular Linked List)

개념:

- 마지막 노드의 링크가 첫 번째 노드를 가리킴
- **NULL**이 없기 때문에 끝을 판단하기 위해 시작점 기준 필요

장점:

- 리스트 처음과 끝 삽입이 단순 연결 리스트보다 쉬움

예시:

- 헤더가 마지막 노드를 가리키도록 구성
- 삽입 시 링크 연결 순서 중요

원형 연결 리스트 응용:

- 운영체제의 Round-Robin CPU 스케줄링
 - 준비 큐, 실행 상태, 대기 상태 반복
 - 각 작업은 고정된 시간 슬롯 단위로 처리

7. 실습 코드 요약 (교재 기준)

- 프로그램 6.9: 연결 리스트 기반 다항식 구현 및 연산
- 프로그램 7.x: 원형 연결 리스트 생성 및 출력 실습

8. 과제 안내

학번 끝자리	수행 과제
0, 1	선택 정렬 (Selection Sort)
2, 3	40점 정렬 과제
4, 5	Double 정렬 알고리즘

과제는 교재의 정렬 알고리즘 단원을 참고하여 구현하고 과제방에 제출.